Semantic Aware Scheduler Explanation Guide

What the current prototype uses, predicts, changes, and measures

This guide explains the working Track A prototype in plain language. It is written for the person presenting and defending the system: what enters the system, what is calculated once, what changes during scheduling, what the learned models predict, and how success is measured. The prototype schedules complete SwinJSCC latent jobs from five simulated user equipments. It does not claim to be a physical 5G MAC scheduler yet.

The most important idea is this: the scheduler is choosing which already-encoded latent job should transmit next. It is not choosing which image is true, classifying disaster images for its own sake, or allocating real radio resource blocks. The image information is converted into scheduling evidence, and that evidence influences the order of whole latent transmissions.

One page summary

Question	Current answer
What is the input?	60 fixed LADI v2 images, their cached semantic features, measured SwinJSCC payload costs, and generated UE arrival/deadline streams.
What is the job?	One complete SwinJSCC latent payload associated with one image and one simulated UE.
What changes at runtime?	The waiting queue, time, deadlines, transport outcome, and delivered-history novelty.
What does a policy output?	One valid candidate index: the next whole latent job to serve.
What does DQN predict?	A Q-value for each possible next job, then it selects the valid job with the largest Q-value.
What is the main score?	Normalized semantic-utility AUC over time, supported by deadline, inversion, fairness, latency, and resource metrics.
What is not implemented?	Real over-air header protection, physical MAC/resource-block allocation, latent-fragment scheduling, and a distributed multi-cell knowledge base.

1 The dataset and what it represents

The current benchmark uses 60 real images from the MITLL LADI v2 disaster-imagery dataset. LADI v2 is multi-label: one image can simultaneously contain roads, buildings, debris, flooding, trees, or other conditions. Therefore the six groups in this prototype are selection strata used to construct a controlled experiment. They are not claimed to be six mutually exclusive natural classes in the original dataset.

Selection stratum	Images	Prototype impact	Prototype urgency	Base priority
flooding_structures	10	3	3	0 Critical
buildings_affected_or_greater	10	3	2	1 Emergency
roads_damage	10	2	3	1 Emergency
debris_any	10	2	2	2 Moderate
trees_damage	10	1	1	2 Moderate
infrastructure_control	10	0	0	3 Normal

The six groups are balanced at ten images each. The 60 images are arranged into 30 two-image evidence groups, with a group-safe split: 36 training images, 12 validation images, and 12 test images. The current prototype deliberately continues with the existing image and pair selection. Its P7 evidence-sufficiency value is defaulted to 1, and the pair relationships are assumptions rather than independent human-reviewed truth.

The source draft is preserved. A separate prototype-fixed manifest is used for execution so that the original data selection is unchanged and the prototype assumption is visible in provenance.

2 What one image becomes

Each image produces two parallel products. The first is a semantic scheduling record. The second is a cached SwinJSCC payload-cost record. The expensive image feature extraction and codec encoding happen once per image; the scheduler does not regenerate or decode the latent for every training episode.

Product	What it contains	Used by
Semantic record	P2 relevance, P3 embedding, P5 impact, P6 urgency, P7 sufficiency, availability bits, provenance, extraction time.	Scheduler input and explanation.
Payload record	Latent shape, storage bytes, measured complex channel uses, encoder time, checkpoint hash and preprocessing.	Service-cost calculation and selected-job replay.
Logical header	A fixed 24-byte V2 control sidecar describing the scheduling fields and receiver history version.	Simulated control metadata; separate from analog JSCC symbols.

3 The five scheduling parameters

Parameter	How it is obtained	Does it change during an episode?
P2 Context relevance	CLIP image embedding cosine similarity with the fixed post-disaster triage task text. Normalized from [-1, 1] to [0, 1].	No. It is cached.
P3 Delivered-history novelty	One minus the maximum cosine similarity with successfully delivered image embeddings, mapped to [0, 1].	Yes. Recomputed when the delivered history changes.
P5 Task impact	Fixed task rubric value from 0 to 3 based on the selected stratum.	No. Prototype metadata.
P6 Urgency	Fixed task rubric value from 0 to 3; it also generates the simulated deadline range.	No. Prototype metadata.
P7 Evidence sufficiency	Prototype default 0 or 1. Current prototype uses 1 for all 60 images.	No. Prototype metadata.

The scheduler receives normalized values and an availability mask. An empty delivered history makes the policy cold-start novelty equal to 1.0, but the availability bit remains false so the model can distinguish “no history measured yet” from a genuinely measured novelty of one.

4 What happens in one episode

An episode is a generated transmission scenario, not a new image dataset. The same cached image catalogue is reused to create many different orderings and UE assignments. This is how the system gets repeated scheduling experience without repeatedly encoding the images.

1. Select 25 jobs for the normal episode. The test tracks use controlled repetition when they need 25 jobs from the 12-image image-disjoint split.
2. Assign jobs across five simulated UEs using seeded traffic profiles. UE identity controls arrival and fairness behaviour; it is not a semantic priority label.
3. Generate an arrival time and a deadline from the job's urgency. The scheduler sees current age and deadline slack, but not future arrivals or future channel outcomes.
4. At a decision point, the simulator exposes an immutable snapshot of waiting jobs. Each job contains semantic values, availability bits, age, slack, service cost, and UE identity.
5. The selected policy chooses one eligible whole latent job. There is no latent-fragment preemption.
6. The simulator advances through service, arrivals, expiration, and the pre-generated transport outcome. Only a successful delivery updates delivered history.
7. The environment calculates the reward and metrics, then the next snapshot recomputes novelty using the new history version.

```
image -> semantic cache + SwinJSCC payload cache -> UE job queue -> policy chooses next job ->
service/outcome -> history update -> next decision
```

5 What each model or policy is predicting

There are several policies because the experiment compares simple network rules, semantic rules, supervised models, and reinforcement learning. They do not all predict the same thing.

Policy	What it sees	What it outputs	Role
FIFO / round robin / EDF / SJF / aging	Queue timing and/or cost.	A deterministic next-job choice.	Classical baselines.
Fixed priority / priority plus EDF	The prototype rubric priority.	A deterministic next-job choice.	Privileged reference baselines.
Semantic greedy	P2, P3, P5, P6, P7 and cost.	A transparent score for each candidate.	Human-readable semantic comparator.
LADI image classifier policy	Predicted LADI multi-label probabilities.	Mapped predicted priority, then next job.	Image-classifier baseline; it never receives ground-truth labels at decision time.
Feature ordinal classifier	Five normalized feature values plus five availability bits.	Expected priority class score 0–3; lowest score first.	Small supervised reference model.
Pairwise ranker	Difference between observable feature rows during training.	A scalar ranking score for each candidate.	Direct relative-order reference model.
Small DQN	25 candidate slots with features, age, slack, cost, UE one-hot, and nine global values.	25 Q-values, one per possible next-job action; invalid actions are masked.	Main learned scheduler.

The DQN is therefore not predicting a disaster class. It predicts the expected long-term value of selecting each currently eligible latent job as the next action. The policy chooses the largest valid Q-value. The output is a sequence of whole-latent scheduling decisions over an episode.

6 How the DQN learns

The DQN uses the standard next-action formulation. State is the current queue snapshot. Action is the index of one eligible job. Reward is generated by the environment after the job's service outcome. The episode ends when jobs reach a terminal state or the safety horizon is reached.

RL concept	This prototype
State	459 values: 25 candidate slots × 18 values, plus nine global queue/time/UE values.
Action	Choose one of 25 candidate positions. Padded and ineligible positions are masked.
Reward	Semantic utility gained, minus service cost, expired-job penalty, and transport-failure penalty.
Episode	One generated 25-job multi-UE transmission scenario.
Training	Standard DQN, target network, replay buffer, Huber loss, epsilon-greedy exploration, three seeds.
Time budget	40 minutes per seed, 2 hours total configured DQN training budget; actual completed episodes are recorded.

The current reward is not a simple match or mismatch. For a successful delivery, the utility uses base-priority weight, evidence-group coverage gain, sufficiency factor, and a delay factor. The transition then subtracts 0.1 times normalized service cost, the normalized weight of newly expired jobs, and 0.25 times the selected job weight when transport fails.

```
reward = 4.0 * semantic_utility_gain - 0.1 * normalized_service_cost - expiry_penalty - failure_penalty
```

The evaluator's priority and evidence-group facts are deliberately hidden from the learned policy observation. They are used by the environment to calculate the teaching signal and by the evaluation code to score the schedule. This

makes the DQN a scheduler trained against a defined task objective, not a model that is given the answer label as an input.

7 What changes after a transmission

Event	Queue/time change	Delivered-history change	Reward consequence
Successful on-time delivery	Selected job leaves queue; time advances.	Record image embedding once; increment history version.	Positive semantic utility, minus service cost.
Successful late delivery	Selected job leaves queue; time advances.	Still record it as delivered; lateness is measured.	Utility is delayed by the delay factor.
Transport failure	Selected job leaves queue; no retransmission in this prototype.	No history update.	Failure penalty and service cost.
Waiting job expires	Job leaves queue as expired.	No history update.	Priority-weighted expiration penalty.
Queue overflow drop	Job never enters the waiting queue.	No history update.	Counted as a drop; no semantic delivery utility.

This success-only history rule is central. Novelty is about what the receiver has successfully received, not what the sender attempted to transmit. A failed, expired, or dropped job must not make related future images look less novel.

8 Dataset evaluation versus scheduler evaluation

There are two different evaluation questions. Mixing them would make the results misleading.

Evaluation layer	Question	Metrics or checks
Dataset and cache quality	Is the 60-image experiment bundle complete, reproducible, and internally consistent?	Exactly 60 unique image IDs; ten per stratum; 30 groups of two; 36/12/12 split; source SHA-256 checks; feature-cache IDs match manifest; payload-cost IDs match manifest; one measured resource unit and checkpoint hash.
Feature extraction	Can the semantic header be produced consistently and quickly?	P2/P3 values, availability masks, extraction latency, CPU/GPU overhead where available, provenance, header size of 24 bytes, CRC round trip.
Static ordering	When all jobs are available at once, does an ordering agree with the rubric?	nDCG, Kendall tau-b, pair agreement, weighted priority inversions. These are only for the static all-at-once fixture.
Dynamic scheduler	Does the policy deliver valuable evidence quickly and fairly under arrivals, deadlines, cost and failures?	Normalized semantic-utility AUC, final utility, critical on-time rate, deadline misses, inversions, coverage, fairness, starvation, service cost, inference latency.
Codec replay	What happens when selected jobs are actually replayed through SwinJSCC?	Measured encoder/decoder latency, latent dimensions, complex channel uses, PSNR/SSIM when valid checkpoint inference succeeds.

There is no single dataset-wide classification accuracy that answers whether the scheduler works. The dataset is a controlled benchmark and source of cached records. The scheduler is evaluated by the quality of the transmission schedule generated from those records.

9 The main scheduler metric

The primary metric is normalized semantic-utility AUC over time. At each successful delivery, the environment adds utility. The utility curve is accumulated over simulated time, then its area is normalized by the episode horizon and the maximum possible utility for that episode.

```
normalized utility AUC = area under accumulated utility curve / (episode horizon × maximum episode utility)
```

This rewards a policy that delivers important and novel evidence early, rather than only eventually. A policy that finishes with the same final utility but delivers critical evidence sooner can therefore have a higher AUC.

Metric	Interpretation	Higher or lower is

		better
Normalized semantic-utility AUC	Primary time-sensitive scheduling quality.	Higher
Final utility	Total accumulated semantic utility by the end.	Higher
Critical/emergency on-time rate	Fraction of priority 0–1 jobs completed by deadline.	Higher
Weighted priority inversion	Severity of serving a lower-priority eligible job before a higher-priority one.	Lower
Distinct evidence-group coverage	How many groups produced delivered evidence.	Higher
Deadline miss rate	Late or expired jobs divided by all jobs.	Lower
Jain fairness	Balance of completed service across UEs.	Higher, interpreted with utility
Maximum starvation time	Worst delivery delay from arrival to terminal outcome.	Lower
Mean/p95 policy latency	Scheduler decision overhead.	Lower
Total resource units	Communication/service cost consumed.	Lower when utility is comparable

10 What counts as scheduler success or failure

The prototype should not be reduced to one binary success flag. Success has several levels, and each level answers a different engineering question.

Level	Success condition	Failure condition
Action validity	The policy chooses an eligible, non-padded candidate index.	Invalid or masked action. This is an implementation failure and should be zero.
Job delivery	The selected job completes on time.	Transport failure, late completion, expiration, overflow drop, or horizon censoring.
Episode behaviour	High normalized utility AUC, strong critical on-time rate, low inversions and acceptable fairness/latency.	Poor utility, missed urgent jobs, starvation, excessive inversions, or unstable inference.
Research comparison	Improves a chosen metric over a baseline on identical streams and transport outcomes, with uncertainty reported.	No improvement, improvement only on one lucky stream, or an unfair comparison.

For tomorrow's demonstration, the clearest claim is comparative: on exactly the same generated scenario, how does the semantic policy's utility-over-time and urgent-delivery behaviour compare with FIFO, EDF, SJF, classifier, ranker and DQN policies? The DQN does not need to win every metric to be a valid experiment. Its result must be reported honestly, including instability or failure to beat the transparent heuristic.

11 Fair comparison rules

- Every policy receives the same immutable episode: same 25 jobs, arrivals, UE assignments, deadlines, payload costs and transport outcome tape.
- The fixed-priority and privileged marginal-utility policies are references with evaluator knowledge. They are not deployable learned policies and should be labelled privileged.
- The LADI classifier policy receives predicted multi-label probabilities, not native dataset ground-truth labels.
- The learned classifier, pairwise ranker and DQN receive only observable feature values and masks. They do not receive base priority, evidence-group ID, future arrivals, or future outcomes.
- The P2/P3-only semantic-greedy result is an automatic-feature comparator. Masking P5/P6/P7 at inference for models trained on all five fields is an input-masking stress test, not a separately retrained automatic-only model.
- Image-disjoint test results and closed-catalogue sequence results must remain separate. The closed-catalogue track tests new order/history behaviour, not unseen-image generalization.

12 What is implemented, simulated and proposed

Status	Current scope

Implemented	Semantic feature records; CLIP P2; decision-time P3; prototype P5/P6/P7; 24-byte V2 header; whole-latent queue simulator; classical policies; classifier; pairwise ranker; small DQN; metrics; two Kaggle notebooks.
Measured when available	SwinJSCC encoding and selected-job replay, latent dimensions, complex channel uses, encoder/decoder timing and reconstruction quality.
Simulated	Five UEs, arrivals, queue capacity, deadlines, service duration, channel success/failure tape, and logical control-header delivery.
Proposed future work	Robust physical control channel, real MAC/resource-block allocation, latent-fragment scheduling, multi-cell operation and distributed knowledge-base synchronization.

13 What you should show in the demonstration

8. Show the 25 incoming image jobs grouped across five UEs.
9. Show one job's P2, P3, P5, P6 and P7 values, availability mask, header bytes and measured payload cost.
10. Show a queue snapshot and explain why each candidate has its current evidence values.
11. Run FIFO, EDF, SJF, semantic greedy, classifier, ranker and DQN on the identical episode.
12. Show the final order, utility-over-time, critical on-time delivery, inversion severity, fairness and decision latency.
13. Select a related-image pair and show P3 changing only after the first image is successfully delivered.
14. Replay a small number of selected latents through the real SwinJSCC checkpoint only if valid checkpoint inference succeeds.
15. End by separating implemented, simulated and proposed parts so the audience knows exactly what the prototype proves.

14 Files to remember

Purpose	File or folder
Dataset package	results/kaggle-ladi60-prototype.zip
Prototype manifest	results/ladi-curation/v1/ladi60_semantic_scheduler_v1_prototype.csv
Feature definitions and normalization	SemParam/scheduler_features.py and INFRA/semantic_contracts.py
History and novelty	SemParam/delivered_history.py and INFRA/event_simulation.py
Reward and metrics	INFRA/semantic_evaluator.py
Policies and DQN inference	Scheduler/semantic_policies.py and Scheduler/feature_adapter.py
DQN training	INFRA/learning/dqn_training.py
Training plus same-run evaluation	Scheduler/notebooks/01_train_semantic_scheduler.ipynb
Independent evaluation rerun	Scheduler/notebooks/02_evaluate_all_policies.ipynb
Operational instructions	docs/SEMANTIC_SCHEDULER_RUNBOOK.md

Final understanding

The prototype takes fixed images, converts each one into semantic scheduling evidence and a cached whole-latent communication job, places those jobs into changing multi-UE queues, and learns or applies policies that choose the next job. The central dynamic signal is delivered-history novelty: successful deliveries change what is new, while failed or undelivered jobs do not. The DQN is successful when it learns valid next-job choices that produce strong time-sensitive semantic utility under the same conditions used by every baseline. The dataset supports that experiment; it is not itself the scheduler's prediction target.

Current implementation provenance: branch semantic-aware-ai-scheduler, latest pushed commit d855cb4. The working tree may contain unrelated local documents and graph artifacts, so the Kaggle notebook intentionally clones the clean pushed commit.